

Resource Allocation Optimization in a Food Bank

Project Report: Memetic Algorithm and Preference Reasoning

Name Surname (Student ID: XXXXXX)

June 8, 2026

Abstract

This report illustrates the development of an Artificial Intelligence system designed to solve the *Food Bank Resource Allocation Problem* (FBRAP). The system employs a memetic approach that hybridizes a Genetic Algorithm (for global optimization) with a greedy post-processing heuristic. Advanced *Preference Reasoning* techniques for priority management and stochastic repair mechanisms based on Softmax distributions have been integrated to maximize fairness and distributional effectiveness in resource-scarce scenarios.

Contents

1	Introduction and Problem Definition	2
2	Data Modeling and Heuristic Priorities	2
3	Architectural and Technological Choices	2
3.1	Database Backend: Supabase	2
3.2	Python and In-Memory Optimization	3
3.3	Decoupling (JSON and CSV Outputs)	3
4	The Genetic Algorithm and Fitness Evaluation	3
5	Smart Repair with Softmax Distribution	3
5.1	Code Analysis	3
6	Memetic Hybridization (Greedy Clean-up)	4
7	Results Analysis and Conclusions	4

1 Introduction and Problem Definition

This project addresses the logistical and humanitarian challenge of distributing limited food resources to a network of beneficiary entities (hospitals, canteens, family homes). The objective is not merely an equal distribution, but rather the maximization of global utility, ensuring that facilities with higher priority and critical needs are satisfied first, while strictly adhering to hard inventory constraints.

2 Data Modeling and Heuristic Priorities

The environment is modeled through three main entities retrieved from the database: *Beneficiary*, *Person*, and *Product*.

Prior to optimization, the system performs a heuristic calculation of priorities. For every single person, a score is calculated based on age, type of hosting facility, caloric requirement, and gender. Subsequently, the global priority of an Entity E is defined as:

$$Priority(E) = \left(\frac{\sum_{p \in E} priority(p)}{|E|} \times w_1 \right) + (|E| \times w_2) \quad (1)$$

This metric guides the entire Artificial Intelligence, allowing it to apply *Preference Reasoning* in its allocation decisions.

3 Architectural and Technological Choices

The development of the system required a careful selection of the technological stack, prioritizing modularity, performance, and software engineering principles.

3.1 Database Backend: Supabase

For persistence and state management (Entities, People, Inventory), **Supabase** was chosen, a *Backend-as-a-Service* (BaaS) based on a PostgreSQL architecture. This choice is motivated by three strategic factors:

- **Rigorous Relational Modeling:** Being an SQL database, Supabase natively handles complex relationships via foreign keys (e.g., the logical association between **Person** and **Beneficiary**), delegating structural JOIN operations directly to the database.
- **API-First Accessibility:** Supabase automatically generates a secure RESTful API over the database. Using the official Python SDK, the AI engine can fetch the entire initial state with a few remote calls, eliminating the need to develop, test, and maintain an intermediate backend server (e.g., in Node.js or Django).
- **Cloud Scalability:** Data is not statically hardcoded into the software, making the application a true microservices architecture ready for real-time operational scenarios.

3.2 Python and In-Memory Optimization

The meta-heuristic optimization engine is written entirely in Python. The choice of language is dictated by its extensive mathematical support and ease of structured data abstraction. To ensure the computational efficiency of the genetic calculation (which requires hundreds of thousands of fitness evaluations), the system adopts an *In-Memory Processing* pattern: data is extracted from Supabase once at startup, translated into Python Data Objects, and indexed via *Hash Maps* (dictionaries). This strategy reduces the time complexity of lookups from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ during the evolutionary loop, preventing bottlenecks caused by I/O or network latency.

3.3 Decoupling (JSON and CSV Outputs)

For the output of results, a deliberate choice was made to avoid destructive writes or direct mutations on the source database. The algorithm merely exports standard artifacts (`.csv` for tabular analysis and `.json` for structured payloads). This ensures **logical decoupling**: the AI solver operates as a pure mathematical module. A potential Frontend development team could simply consume the generated JSON file to render loading plans for delivery vans in a graphical interface, guaranteeing high cohesion and low coupling.

4 The Genetic Algorithm and Fitness Evaluation

The core of the search engine is a Genetic Algorithm. Each chromosome represents a complete mapping of allocations ($\text{Entity} \rightarrow \text{Product} \rightarrow \text{Quantity}$). The fitness function employs a *Penalty Method* that severely penalizes warehouse violations (*Hard Constraints*) and sanctions nutritional imbalances (*Soft Constraints*).

Crucial for solving the FBRAP is the implementation of the **Minimum Threshold** (α -**threshold**), set at 20% of the caloric requirement. If an entity receives food but does not surpass this survival threshold, the function applies a combined penalty (`unserved` + `wasted_food`), inhibiting the ineffective dispersion of resources ("sprinkling effect") in favor of concentration on high-priority targets.

5 Smart Repair with Softmax Distribution

During evolution, solutions may generate physically impossible distributions (inventory excess). To correct these aberrations without destroying genetic diversity, a smart *Repair* operator based on a **Softmax** distribution was designed.

Instead of subtracting excess food in a purely random manner, the algorithm calculates a penalization probability inversely proportional to the entity's urgency. The applied mathematical formula is:

$$W_i = \exp \left(-P_i \times \frac{F_i^{\text{missing}}}{F_i^{\text{total}}} \right) \quad (2)$$

5.1 Code Analysis

The algorithm (Listing ??) has been engineered to maximize computational performance while maintaining complex decision-making logic:

1. **Pre-calculation of Support Maps (Lines 1-5):** Since the `while` loop is executed hundreds of times, recalculating the caloric intake of each entity inside the loop would cause a performance collapse. The currently possessed calories are calculated once before the loop and stored in `alloc_cals_map`.
2. **Zero-Division Handling (Lines 13-16):** To ensure mathematical stability, the `fabbisogno_tot` variable is protected with `max(1, ...)`. Similarly, `fabbisogno_mancante` is floored at zero (`max(0, ...)`) to prevent an over-allocated entity from generating negative values that would distort the exponential.
3. **Softmax Calculation (Lines 18-21):** The exponent is calculated. Being the product of positive quantities preceded by a minus sign, the exponent will always be non-positive ($x \leq 0$). Applying `math.exp(esponente)` will therefore yield a weight $W \in (0, 1]$. Entities with already satisfied requirements (missing requirement close to 0) will have an exponent of 0 and, consequently, a maximum weight ($e^0 = 1$), making them the prime candidates for food removal.
4. **Dynamic $\mathcal{O}(1)$ Update (Lines 27-29):** Once the penalized `target_ben` is drawn, one unit of the product is removed from the chromosome. Simultaneously, the total calories in its possession (`alloc_cals_map`) are subtractively reduced by the exact value of the product (`prod_cal`). This constant-time operation allows the algorithm to perfectly re-evaluate the Softmax weights in the next iteration without performing costly recalculations.

6 Memetic Hybridization (Greedy Clean-up)

After the genetic evolution is complete, the optimal solution might leave small residual stocks in the warehouse. The algorithm transforms into a **Memetic Algorithm** by applying a purely deterministic *Local Search* phase. The algorithm assigns the remnants in descending order of priority to the entities that still exhibit a caloric gap, introducing a logical *Hard Constraint*: it is strictly forbidden to exceed 100% of an entity's requirement, effectively eliminating physiological waste.

7 Results Analysis and Conclusions

The execution of the system generates advanced analytical metrics. The distributional charts highlight the *Triage Effect* operated by the AI: in cases of severe scarcity, the algorithm rapidly saturates the highest-priority facilities, partially sacrificing lower-priority entities to avoid the heavy penalties established by the *Preference Reasoning*.

The developed model demonstrates how the combination of Genetic Algorithms, local heuristics, and preference-based mathematical reasoning (via Penalty Method and Softmax functions) offers a robust and computationally efficient solution for NP-Hard problems in the field of humanitarian logistics.